

MANUAL AVANZADO DE SQL

Estudio detallado de estructuras y lógica relacional aplicado al entorno SonoraDB.

1. Consultas Básicas: SELECT, WHERE y ORDER BY

La recuperación de información es el pilar de SQL. Una consulta básica no solo extrae datos, sino que sigue un orden de ejecución lógico interno en el motor de la base de datos que difiere del orden en el que se escribe:

- **FROM (Origen):** El motor primero localiza la tabla o tablas base.
- **WHERE (Filtro):** Se evalúa cada fila frente a una condición lógica. Aquellas que retornan *TRUE* pasan a la siguiente fase; las de valor *FALSE* o *NULL* se descartan.
- **SELECT (Proyección):** Se determinan qué columnas específicas se van a devolver. Aquí se resuelven los alias (AS).
- **ORDER BY (Ordenación):** Finalmente, el subconjunto filtrado y proyectado se ordena. Es la única cláusula que garantiza el orden de salida (por defecto ascendente ASC).

EJEMPLO A: FILTRADO Y ORDENACIÓN SIMPLE

Ejercicio 4

Mostrar el nombre de usuario, el plan y la fecha de alta de los usuarios de España (ES), del más antiguo al más reciente.

```
SELECT nombre_usuario, plan_suscripcion, fecha_alta FROM usuarios WHERE pais = 'ES' ORDER BY fecha_alta ASC;
```

Análisis: El filtro WHERE discrimina los registros cuyo campo pais coincide exactamente con la cadena literal 'ES'. Posteriormente, ORDER BY fecha_alta ASC asegura que el registro con la fecha más antigua se sitúe en la primera posición del conjunto de resultados.

EJEMPLO B: FILTRADO DE RANGOS OPTIMIZADOS

Ejercicio 5

Buscar canciones lanzadas en 2026 que duren entre 170 y 200 segundos. Filtrar el año con rango de fechas sin aplicar funciones sobre la columna.

```
SELECT titulo, duracion_seg, fecha_lanzamiento FROM canciones WHERE duracion_seg BETWEEN 170 AND 200 AND fecha_lanzamiento >= '2026-01-01' AND fecha_lanzamiento < '2027-01-01' ORDER BY fecha_lanzamiento;
```

Análisis: BETWEEN incluye ambos extremos (170 y 200). Para el año, en lugar de usar YEAR(fecha_lanzamiento) = 2026, se usa un rango >= y <. Esto se conoce como una condición **SARGable** (Search Argument Able): al no aplicar funciones matemáticas sobre la columna base, permitimos al motor de base de datos utilizar índices de forma eficiente.

2. Expresiones y Funciones

SQL permite transformar, formatear o categorizar los datos en "tiempo de vuelo" (mientras se consultan, sin modificar la tabla). Las funciones se ejecutan por cada fila del conjunto de resultados.

- **Expresión CASE:** Es el análogo al if-else en programación. Evalúa condiciones en cascada de arriba a abajo.
- **Funciones de Cadenas:** CONCAT () enlaza textos sin fallar ante nulos. Alternativamente, el estándar SQL usa el operador ||.
- **Conversión de Tipos (CAST / CONVERT):** Permite alterar el tipo de dato subyacente para mostrarlo o procesarlo distinto (ej. aislar la hora de un campo Fecha-Hora).

EJEMPLO A: LÓGICA CONDICIONAL CON CASE

Ejercicio 11

Clasificar las canciones de Nébula (artista_id = 5) por duración: Corta (<180s), Media (180-240s) y Larga (>240s).

```
SELECT titulo, duracion_seg, CASE WHEN duracion_seg < 180 THEN 'Corta' WHEN
duracion_seg <= 240 THEN 'Media' ELSE 'Larga' END AS categoria FROM canciones WHERE
artista_id = 5 ORDER BY titulo;
```

Análisis: Al usar CASE, el motor evalúa secuencialmente. Si una canción dura 150 segundos, entra por la primera condición ('Corta') y sale del bloque. Por eso en la segunda condición basta con validar <= 240, ya que todas las menores de 180 fueron atrapadas arriba.

EJEMPLO B: FORMATEO Y EXTRACCIÓN (CAST)

Ejercicio 14

Para las reproducciones del usuario 3, separar el campo fecha_hora en dos columnas independientes: fecha (DATE) y hora sin fracciones (TIME(0)).

```
SELECT reproduccion_id, CAST(fecha_hora AS DATE) AS fecha, CAST(fecha_hora AS
TIME(0)) AS hora, segundos_escuchados FROM reproducciones WHERE usuario_id = 3 ORDER BY
fecha, hora;
```

Análisis: El tipo DATETIME2 almacena la línea de tiempo completa. Usando la función de casteo CAST(columna AS TIPO_NUEVO) forzamos al motor a mutilar la información que no necesitamos. TIME(0) específicamente elimina los milisegundos de la visualización horaria.

💡 **Tip sobre Nulos (NULL):** En SQL, NULL significa "desconocido", no "vacío". Una comparación como `cancion_id = NULL` siempre devuelve falso (o desconocido), porque algo desconocido no es igual a otra cosa desconocida. La sintaxis correcta es siempre `IS NULL` o `IS NOT NULL`.

3. Agregación: GROUP BY y HAVING

Las funciones de agregación (SUM, COUNT, AVG, MAX, MIN) procesan un bloque de filas múltiples y retornan un único escalar. Cuando las combinamos con GROUP BY, fragmentamos la tabla original en "cubos" basados en valores comunes y ejecutamos las matemáticas sobre cada cubo.

- **COUNT(*):** Cuenta filas absolutas del cubo, incluyendo las que tienen valores NULL.
- **COUNT(columna):** Cuenta cuántos valores NO NULOS existen en esa columna específica.
- **HAVING:** Es el WHERE de los grupos. Mientras que `WHERE` descarta filas individuales antes de agrupar, `HAVING` descarta "cubos" enteros después de que se han calculado las sumas o promedios.

EJEMPLO A: AGRUPACIÓN Y MATHS DE PRECISIÓN

Ejercicio 17

Por cada dispositivo, obtener el total de reproducciones, total de segundos y total de minutos con un único decimal.

```
SELECT dispositivo, COUNT(*) AS reproducciones, SUM(segundos_escuchados) AS segundos, CAST(SUM(segundos_escuchados) / 60.0 AS DECIMAL(10,1)) AS minutos FROM reproducciones GROUP BY dispositivo ORDER BY reproducciones DESC;
```

Análisis: GROUP BY dispositivo crea 4 cubos (Escritorio, Móvil, Smart TV, Web). Al dividir por 60.0 forzamos una conversión implícita a decimal antes de realizar la división; si dividiéramos por el entero 60, SQL Server truncaría la parte fraccionaria. Finalmente, CAST AS DECIMAL(10,1) redondea la salida visual a un decimal.

EJEMPLO B: FILTRADO POST-AGREGACIÓN CON HAVING

Ejercicio 19

Marketing busca "superoyentes": usuarios con más de 3 reproducciones tipo 'Canción' y más de 800 segundos escuchados en total.

```
SELECT u.nombre_usuario, COUNT(*) AS reproducciones, SUM(r.segundos_escuchados) AS segundos FROM reproducciones r INNER JOIN usuarios u ON r.usuario_id = u.usuario_id WHERE r.tipo_contenido = 'Canción' GROUP BY u.nombre_usuario HAVING COUNT(*) > 3 AND SUM(r.segundos_escuchados) > 800 ORDER BY segundos DESC;
```

Análisis: El WHERE actúa primero ignorando reproducciones de 'Anuncios'. Luego se agrupa por usuario. Finalmente, HAVING verifica las métricas matemáticas del grupo resultante, permitiendo usar las funciones agregadas directamente en la condición.

⚠ Regla de Oro del GROUP BY: Cualquier columna que aparezca en el SELECT y no esté envuelta en una función de agregación (como COUNT o SUM), **debe** aparecer explícitamente en la cláusula GROUP BY.

4. JOIN y UNION (Álgebra Relacional)

La normalización de bases de datos divide los datos en múltiples tablas para evitar redundancias. Los JOINs son los mecanismos para reconstruir esas relaciones combinando filas horizontalmente en función de claves coincidentes, mientras que UNION apila conjuntos enteros de forma vertical.

EJEMPLO A: RECONSTRUCCIÓN MULTI-TABLA (INNER)

Ejercicio 21

Listar canciones de artistas españoles con el nombre del artista y el nombre del género de la canción.

```
SELECT a.nombre AS artista, c.titulo, g.nombre AS genero FROM canciones c INNER JOIN artistas a ON c.artista_id = a.artista_id INNER JOIN generos g ON c.genero_id = g.genero_id WHERE a.pais = 'ES' ORDER BY a.nombre, c.titulo;
```

Construcción paso a paso:

1. **Definir la base:** FROM canciones c. Empezamos en la tabla central (la que tiene las FKs a las demás) y le asignamos el alias corto 'c'.
2. **Primera conexión:** INNER JOIN artistas a. Añadimos los artistas, y la cláusula ON c.artista_id = a.artista_id sirve de "pegamento" físico entre ambas tablas.
3. **Segunda conexión:** INNER JOIN generos g. Usamos la misma técnica para traducir el ID del género a su texto legible.
4. **Proyección y alias:** Subimos al SELECT. Al haber campos repetidos en las tablas (ej: "nombre" existe en artistas y géneros), es obligatorio usar el prefijo a.nombre o g.nombre y renombrarlos con AS.
5. **Filtro final:** Aplicamos el criterio geográfico (WHERE a.pais = 'ES') y ordenamos alfabéticamente.

EJEMPLO B: EL PATRÓN ANTI-JOIN (LEFT + IS NULL)

Ejercicio 24

Obtener los usuarios registrados que jamás han hecho ninguna reproducción.

```
SELECT u.nombre_usuario, u.plan_suscripcion, u.fecha_alta FROM usuarios u LEFT JOIN reproducciones r ON u.usuario_id = r.usuario_id WHERE r.reproduccion_id IS NULL;
```

Construcción paso a paso:

1. **Preservar el conjunto inicial:** FROM usuarios u. Colocamos la tabla de la que no queremos perder registros a la izquierda.
2. **Cruce inclusivo:** LEFT JOIN reproducciones r. A diferencia del INNER, el LEFT JOIN garantiza que si un usuario NO tiene reproducciones, seguirá apareciendo en la lista, pero todas las columnas de la derecha (las de la tabla 'r') se rellenarán con valores nulos artificiales.
3. **Encontrar los vacíos:** WHERE r.reproduccion_id IS NULL. Usamos la clave primaria de la tabla derecha. Al buscar aquellas filas donde esta clave sea nula, estamos filtrando matemáticamente y quedándonos solo con los usuarios inactivos.

5. DDL: Lenguaje de Definición de Datos

DDL (CREATE, ALTER, DROP) construye el "chasis" físico de la base de datos. Define las tablas y las restricciones de dominio que actúan como barreras de seguridad para los datos.

EJEMPLO A: CREAR UNA TABLA DEFENSIVA

Ejercicio 29

```
CREATE TABLE dbo.playlists ( playlist_id INT IDENTITY(1,1) CONSTRAINT
PK_playlists PRIMARY KEY, usuario_id INT NOT NULL CONSTRAINT FK_plyls_usr FOREIGN KEY
REFERENCES usuarios(usuario_id), nombre NVARCHAR(100) NOT NULL, es_publica BIT NOT NULL
CONSTRAINT DF_plyls_pub DEFAULT 0, fecha_creacion DATETIME2(0) NOT NULL CONSTRAINT
DF_plyls_fch DEFAULT SYSDATETIME(), CONSTRAINT UQ_usuario_nombre UNIQUE (usuario_id,
nombre) );
```

Construcción paso a paso:

1. **Identidad y Clave Primaria:** `playlist_id INT IDENTITY(1,1)`. Declaramos el tipo numérico. La propiedad `IDENTITY` instruye al motor a autoincrementar este valor (empieza en 1, suma 1), liberando a la app de tener que enviarlo. Se cierra declarándolo como `PRIMARY KEY`.
2. **Claves Foráneas (FK):** `usuario_id` se declara. Luego, con `FOREIGN KEY REFERENCES usuarios(usuario_id)`, establecemos una regla de hierro: es imposible guardar una playlist cuyo `usuario_id` no exista previamente en la tabla de usuarios.
3. **Valores por defecto:** Añadir `DEFAULT 0` (falso) y `DEFAULT SYSDATETIME()` (fecha actual) permite que las consultas `INSERT` sean más cortas, delegando estos rellenos a la base de datos.
4. **Reglas Combinadas:** `UNIQUE (usuario_id, nombre)`. Esta restricción se declara al final (a nivel de tabla) porque cruza dos columnas. Evita que un mismo usuario tenga dos listas llamadas exactamente igual, aunque distintos usuarios sí pueden repetir nombres.

EJEMPLO B: ALTERACIÓN ITERATIVA (ALTER)

Ejercicio 31

```
ALTER TABLE dbo.playlists ADD descripcion NVARCHAR(200) NULL; ALTER TABLE
dbo.playlists ADD CONSTRAINT CK_play_nombre CHECK (LEN(nombre) >= 3);
```

Construcción paso a paso:

1. **Modificar Columnas:** `ADD descripcion`. Es obligatorio que al añadir una columna en un entorno en producción (que ya tiene filas previas), la nueva columna permita nulos (`NULL`). Si fuese `NOT NULL`, el comando fallaría al no saber qué poner en las filas antiguas.
2. **Reglas a nivel fila (CHECK):** `CHECK (LEN(nombre) >= 3)` instruye al motor a ejecutar esta función matemática por cada futuro `INSERT` o `UPDATE`. Si alguien intenta guardar el nombre "AB", la transacción será abortada instantáneamente por el motor (Error 547).

6. DML: Lenguaje de Manipulación de Datos

DML (INSERT, UPDATE, DELETE) opera sobre los datos reales. Toda instrucción DML está sujeta a la validación estricta de las reglas DDL; si se viola una regla, toda la transacción se revierte de inmediato.

EJEMPLO A: INSERCIÓN DINÁMICA MASIVA

Ejercicio 33

A la playlist 3, inyectar automáticamente todas las canciones de géneros 'House' y 'Techno'.

```
INSERT INTO playlist_canciones (playlist_id, cancion_id) SELECT 3, c.cancion_id
FROM canciones c INNER JOIN generos g ON c.genero_id = g.genero_id WHERE g.nombre IN
('House', 'Techno');
```

Construcción paso a paso:

1. **El cascarón de inserción:** INSERT INTO [...] (playlist, cancion). Definimos explícitamente en qué orden vamos a proporcionar las columnas a la base de datos.
2. **Generar los datos:** En lugar de insertar filas "a mano" con VALUES, montamos una subconsulta con un JOIN completo. La clave está en el SELECT 3, c.cancion_id. Esto genera al vuelo una tabla virtual que repite el número '3' en la primera columna, seguido del ID de cada canción de House/Techno que encontró.
3. **Volcado automático:** Al ejecutar el bloque, el motor toma esas decenas de filas virtuales generadas por el SELECT y las incrusta de golpe en la tabla de destino, de forma limpia y masiva.

EJEMPLO B: ACTUALIZACIÓN TRANSVERSAL (JOIN-UPDATE)

Ejercicio 35

```
UPDATE p SET p.es_publica = 1 FROM playlists p INNER JOIN usuarios u ON
p.usuario_id = u.usuario_id WHERE u.plan_suscripcion = 'Premium';
```

Construcción paso a paso:

1. **Declarar el cambio:** UPDATE p SET p.es_publica = 1. Indicamos al motor el alias de la tabla destino y la nueva mutación del estado a realizar.
2. **El contexto externo:** En SQL Server podemos inyectar un FROM ... JOIN a un UPDATE. Cruzamos las playlists con los usuarios. Esto permite que el filtro (WHERE plan = 'Premium') opere sobre datos externos (el usuario) para tomar la decisión de modificar el objeto actual (la playlist).

Manejo Transaccional de Errores (DELETE)

Acción SQL	Respuesta Motor	Motivo Técnico
DELETE FROM playlists WHERE id=3	Error 547 (Foreign Key)	Falla porque playlist_canciones tiene canciones colgando de esta lista. Borrar la cabecera rompería la cascada relacional.
DELETE FROM playlist_canciones luego... DELETE FROM playlists	Correcto (Rows affected)	Orden relacional limpio: primero se purgan los registros "hijos" (detalle), para finalmente eliminar la entidad "padre" sin provocar conflictos.